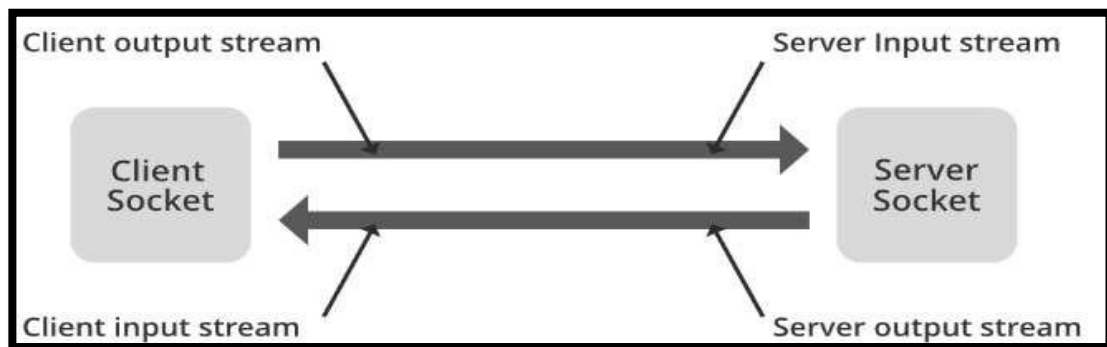


## Practical – 6

|                                                                                                                                                            |                            |
|------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------|
| <b>Government College of Engineering, Jalgaon</b><br><b>(An Autonomous Institute of Govt. of Maharashtra)</b><br><b>Department of Computer Engineering</b> |                            |
| <b>Name:</b>                                                                                                                                               |                            |
| <b>PRN:</b>                                                                                                                                                | <b>Batch:</b>              |
| <b>Class:</b>                                                                                                                                              | <b>Semester:</b>           |
| <b>Course:</b>                                                                                                                                             | <b>Subject Teacher:</b>    |
| <b>Date of Performance:</b>                                                                                                                                | <b>Date of Completion:</b> |

**Aim :** Program for Distributed chat server using TCP sockets.

### Theory :



As computer science students, understanding networking concepts and communication protocols is crucial. In this tutorial, we'll explore how to establish a TCP connection between two servers using Java. We'll build a simple example where a client and a server exchange messages over the TCP protocol.

### 1. Introduction to TCP Connection

TCP (Transmission Control Protocol) is a reliable and connection-oriented protocol used for data communication between devices over a network. It guarantees the delivery of data in the order it was sent and handles any lost or corrupted packets. This makes it suitable for applications where data integrity and order are important, such as file transfers, web browsing, and more.

### 2. What is a Socket?

A socket is a fundamental concept in computer networking that serves as an endpoint for sending or receiving data across a computer network. It's a software interface that enables communication between two devices over a network, allowing data to flow between them. Sockets are a crucial building block for creating networked applications, such as client-server systems, where devices exchange information. Sockets can be thought of as the interface through which programs interact with the network. They abstract the complexity of network communication and provide a simple and consistent way for applications to transmit and receive data.

### 3. Setting Up Server and Client

1. At the heart of our TCP client lies Java's standard library, with its networking and I/O packages. Importing these packages is our first step toward building the client. The `java.io.IOException` package handles exceptions related to input and output operations, while `java.net.InetSocketAddress` and `java.net.Socket` packages provide the tools necessary for networking.
2. Sockets serve as the foundation of network communication, enabling data transfer between devices. We instantiate a `ServerSocket` on port 5001 in `TCPServer.java` and a `Socket` in `TCPClient.java`.
3. The main difference between a socket and a server socket is their role in communication. A socket can be used on both the client and server sides of a networked application. A server socket (e.g., a `ServerSocket` instance in Java) is a specialized socket that listens for incoming connections from clients. It's specifically used by server applications to **accept** new client connections.
4. The `connect` function takes two arguments:  
an `InetSocketAddress` representing the server's IP address and port, and a timeout in milliseconds. In our example, we're attempting to connect to the server with IP address "127.0.0.1" (localhost) on port 5001. The timeout is set to 1000 milliseconds (1 second).
5. Compile and Run  
Left Side is the Server Side and Right Side is the Client Side. The last line prints a confirmation message to the console, letting us know that the client successfully connected to the server.

### 4. Passing Messages Between Servers using Java's `DataInputStream` and `DataOutputStream`

In the realm of networking, the ability to exchange data between servers is a fundamental aspect. Java provides powerful tools to achieve this. We will explore how to establish a communication

channel between two servers using the `DataInputStream` and `DataOutputStream` classes. These classes simplify the process of sending and receiving data over a network connection. We will walk through an example of a client-server interaction where messages are passed back and forth using TCP sockets. Here is the updated Server and Client Code

1. We create a `DataInputStream` and `DataOutputStream` to handle communication with the server.
2. We send a message to the server using `dataOut.writeUTF()`.
3. We read the server's response using `dataIn.readUTF()` ().
4. In both the server and client code examples provided, you might have noticed the use of the `socket.close()` method. This method is crucial for ensuring that the network connection is properly closed after communication is complete. When you're finished exchanging data with a client or server, calling `close()` on the socket helps release resources associated with the connection.
5. Compile and Run

**Code :**

## Server.java

```
import socket
import threading

# Server class to handle client connections
class ChatServer:
    def __init__(self, host='127.0.0.1', port=65432):
        self.host = host
        self.port = port
        self.server_socket = None
        self.clients = [] # List to keep track of connected clients
        self.client_addresses = {} # Dictionary to store client address and socket

    def start(self):
        # Create a TCP/IP socket
        self.server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        self.server_socket.bind((self.host, self.port))
        self.server_socket.listen(5) # Max number of queued connections
        print(f'Server started on {self.host}:{self.port}')

        # Accept incoming connections
        while True:
            client_socket, client_address = self.server_socket.accept()
            print(f'New connection from {client_address}')
            self.clients.append(client_socket)
            self.client_addresses[client_socket] = client_address

            # Start a new thread to handle the client
            client_thread = threading.Thread(target=self.handle_client, args=(client_socket,))
            client_thread.start()

    def handle_client(self, client_socket):
        # Handle communication with a single client
        try:
            client_socket.send("Welcome to the chat server!".encode('utf-8'))

            while True:
                # Receive message from the client
                message = client_socket.recv(1024).decode('utf-8')
                if not message:
                    break # Client disconnected

                # Broadcast the message to all connected clients
                print(f'Message from {self.client_addresses[client_socket]}: {message}')
                self.broadcast_message(message, client_socket)
```

```

except Exception as e:
    print(f'Error handling client {self.client_addresses.get(client_socket)}: {e}')
finally:
    # Cleanup when client disconnects
    self.clients.remove(client_socket)
    del self.client_addresses[client_socket]
    client_socket.close()

def broadcast_message(self, message, sender_socket):
    # Broadcast message to all clients except the sender
    for client_socket in self.clients:
        if client_socket != sender_socket:
            try:
                client_socket.send(message.encode('utf-8'))
            except Exception as e:
                print(f'Error broadcasting to {client_socket}: {e}')

if __name__ == "__main__":
    chat_server = ChatServer()
    chat_server.start()

```

## Client.java

```

import socket
import threading

# Client class to interact with the chat server
class ChatClient:
    def __init__(self, host='127.0.0.1', port=65432):
        self.host = host
        self.port = port
        self.client_socket = None

    def start(self):
        # Create a TCP/IP socket and connect to the server
        self.client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        self.client_socket.connect((self.host, self.port))
        print(f'Connected to chat server at {self.host}:{self.port}')

        # Start a thread to listen for incoming messages
        receive_thread = threading.Thread(target=self.receive_messages)
        receive_thread.daemon = True
        receive_thread.start()

```

```

        # Main thread will handle sending messages
        self.send_messages()

    def receive_messages(self):
        # Continuously listen for messages from the server
        while True:
            message = self.client_socket.recv(1024).decode('utf-8')
            if message:
                print(f"\n{message}")

    def send_messages(self):
        # Continuously prompt the user to send messages
        while True:
            message = input("> ")
            if message:
                self.client_socket.send(message.encode('utf-8'))

if __name__ == "__main__":
    chat_client = ChatClient()
    chat_client.start()

```

**Output :**

### Client-

```
D:\Me\College\DSL\6_RMI>python Client.py
Connected to chat server at 127.0.0.1:65432
>
Welcome to the chat server!
Hello
> I am Client
> Hello World !
> Have a nice day !
> Byeee
> See you
> █
```

### Server-

```
D:\Me\College\DSL\6_RMI>python Server.py
Server started on 127.0.0.1:65432
New connection from ('127.0.0.1', 65362)
Message from ('127.0.0.1', 65362): Hello
Message from ('127.0.0.1', 65362): I am Client
Message from ('127.0.0.1', 65362): Hello World !
Message from ('127.0.0.1', 65362): Have a nice day !
Message from ('127.0.0.1', 65362): Byeee
Message from ('127.0.0.1', 65362): See you
█
```

**Conclusion:** In this practical we learnt about implementation of distributed chat application using TCP sockets.

**Sign of Teacher**  
**Mrs. Archana Chitte**